

LAPORAN PRAKTIKUM ASD



Muhammad Saum Saifulloh

254107060155

Sistem Informasi Bisnis / 1E

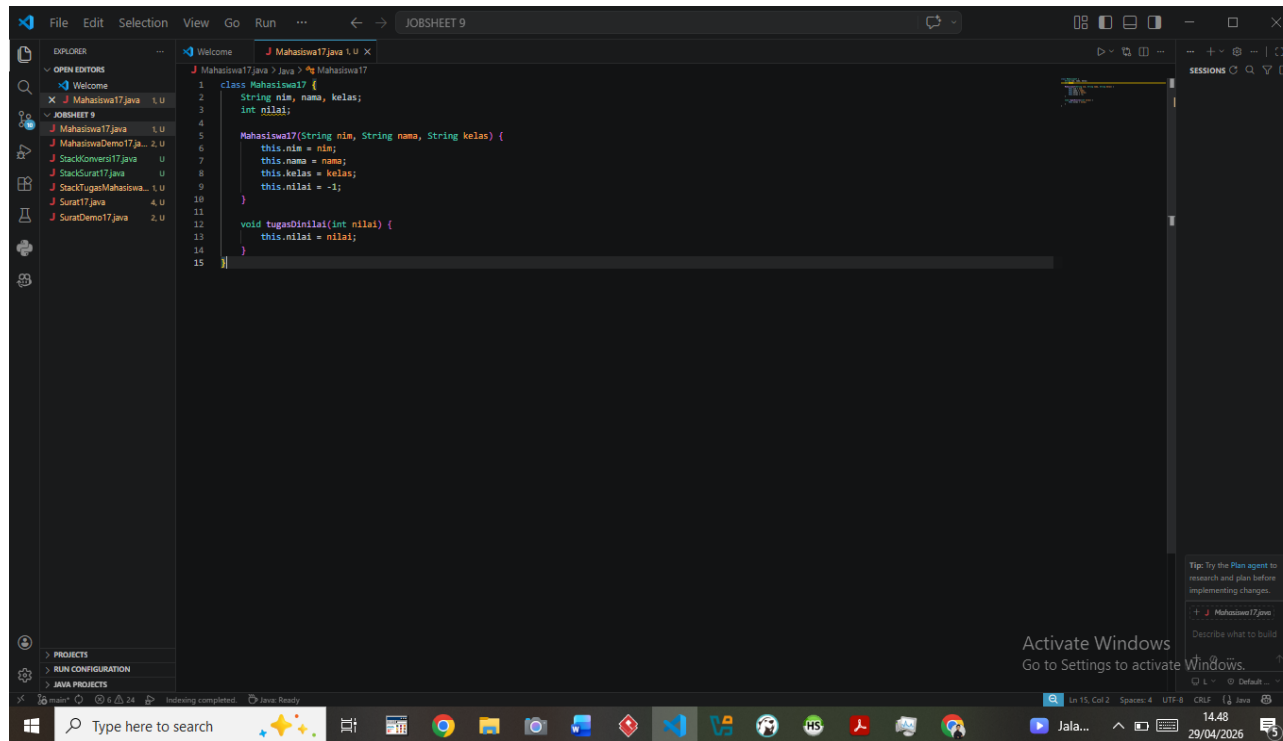
JOBSHEET 9 STACK

Tujuan Praktikum

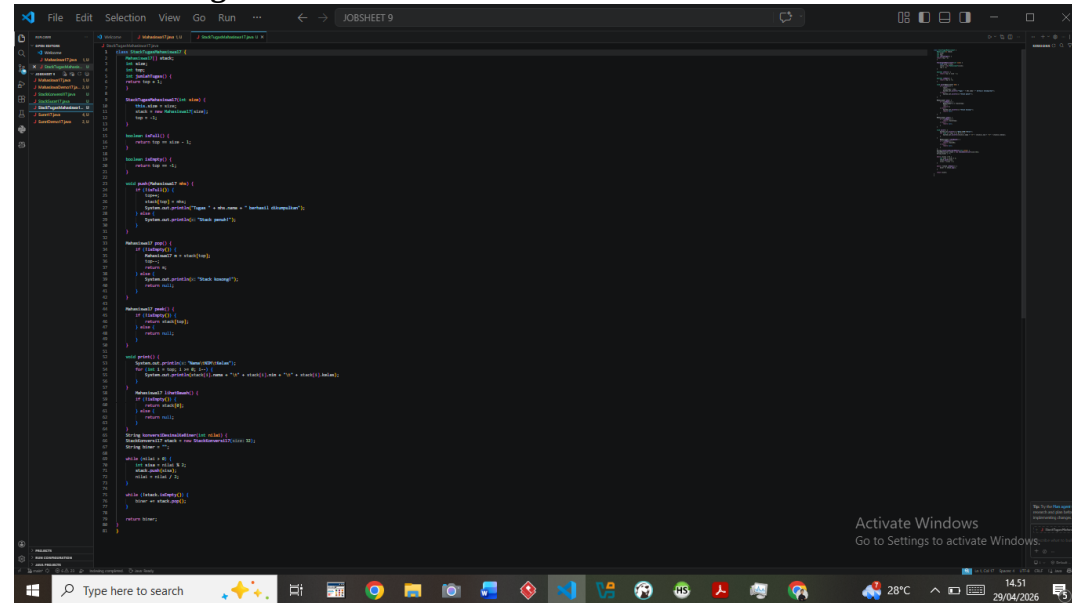
1. Membuat struktur data Stack.
2. Menerapkan algoritma Stack dalam program Java.
3. Mengimplementasikan operasi push, pop, peek, print.
4. Mengonversi nilai desimal ke biner menggunakan stack

PERCOBAAN 1 – Mahasiswa Mengumpulkan Tugas

Class Mahasiswa:

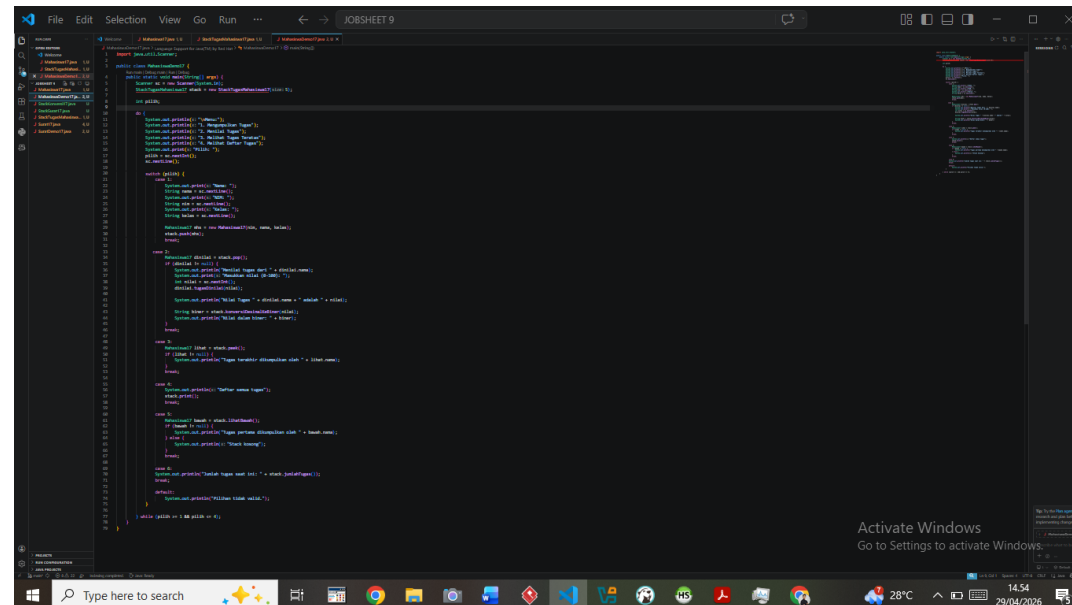


Class Stack Tugas Mahasiswa:



```
1 // Stack.h
2 #ifndef STACK_H
3 #define STACK_H
4
5 #include <iostream>
6
7 class Stack {
8     private:
9         int* arr;
10        int top;
11
12    public:
13        Stack(int size);
14        ~Stack();
15        void push(int data);
16        int pop();
17        bool isEmpty();
18        void display();
19};
20
21 // Stack.cpp
22 #include "Stack.h"
23
24 Stack::Stack(int size) {
25     arr = new int[size];
26     top = -1;
27 }
28
29 Stack::~Stack() {
30     delete[] arr;
31 }
32
33 void Stack::push(int data) {
34     if (top < arr.size() - 1) {
35         arr[++top] = data;
36     } else {
37         std::cout << "Stack is full\n";
38     }
39 }
40
41 int Stack::pop() {
42     if (top >= 0) {
43         return arr[top--];
44     } else {
45         std::cout << "Stack is empty\n";
46         return -1;
47     }
48 }
49
50 bool Stack::isEmpty() {
51     return top <= 0;
52 }
53
54 void Stack::display() {
55     if (top <= 0) {
56         std::cout << "Stack is empty\n";
57     } else {
58         for (int i = 0; i <= top; i++) {
59             std::cout << arr[i] << " ";
60             if (i % 5 == 4) std::cout << "\n";
61         }
62     }
63 }
64
65 int main() {
66     Stack s(10);
67     s.push(10);
68     s.push(20);
69     s.push(30);
70     s.push(40);
71     s.push(50);
72     s.display();
73     s.pop();
74     s.pop();
75     s.pop();
76     s.pop();
77     s.pop();
78     s.display();
79     return 0;
80 }
```

Class Mahasiswa Demo:



```
1 // Mahasiswa.h
2 #ifndef MAHASISWA_H
3 #define MAHASISWA_H
4
5 #include <iostream>
6
7 class Mahasiswa {
8     private:
9         std::string nama;
10        int nim;
11
12    public:
13        Mahasiswa();
14        Mahasiswa(std::string n, int ni);
15        ~Mahasiswa();
16        std::string getNama();
17        int getNIM();
18        void display();
19};
20
21 // Mahasiswa.cpp
22 #include "Mahasiswa.h"
23
24 Mahasiswa::Mahasiswa() {
25     nama = "";
26     nim = 0;
27 }
28
29 Mahasiswa::Mahasiswa(std::string n, int ni) {
30     nama = n;
31     nim = ni;
32 }
33
34 Mahasiswa::~Mahasiswa() {
35 }
36
37 std::string Mahasiswa::getNama() {
38     return nama;
39 }
40
41 int Mahasiswa::getNIM() {
42     return nim;
43 }
44
45 void Mahasiswa::display() {
46     std::cout << "Nama: " << nama << "\n";
47     std::cout << "NIM: " << nim << "\n";
48 }
49
50 int main() {
51     Mahasiswa m1;
52     m1.display();
53     m1 = Mahasiswa("Budi", 123456);
54     m1.display();
55     return 0;
56 }
```

[illegible]

Program dibuat untuk simulasi pengumpulan tugas mahasiswa dengan konsep LIFO (Last In First Out).

- mahasiswa yang terakhir mengumpulkan tugas → dinilai lebih dulu.

- Mengumpulkan tugas (push)
- Menilai tugas (pop)
- Lihat tugas teratas (peek)
- Tampilkan semua tugas (print)
- Lihat tugas pertama (lihatBawah)
- Hitung jumlah tugas (jumlahTugas)

1. Bagian mana yang perlu diperbaiki?

- rename file MahasiswaDemo17,.java menjadi MahasiswaDemo17.java
- memperbaiki typo class:
 - dari StackTugastMahasiswa17
 - menjadi StackTugasMahasiswa17

Agar nama file dan class sesuai aturan Java.

2. Berapa banyak data yang ditampung?

Stack menampung **5 data**.

Potongan kode:

```
StackTugasMahasiswa17 stack = new StackTugasMahasiswa17(5);
```

Angka 5 menunjukkan kapasitas stack.

3. Mengapa perlu `!isFull()` pada `push`?

Agar data hanya ditambahkan saat stack belum penuh.

Potongan kode:

```
if (!isFull()) {  
    top++;  
    stack[top] = mhs;  
}
```

Jika dihapus:

- array bisa melebihi batas
 - muncul error `ArrayIndexOutOfBoundsException`
-

4. Modifikasi lihat tugas terbawah

Ditambahkan method:

```
Mahasiswa17 lihatBawah() {  
    return stack[0];  
}
```

Method ini menampilkan mahasiswa pertama yang mengumpulkan tugas.

Karena `stack[0]` adalah data pertama.

5. Menghitung jumlah tugas

Method:

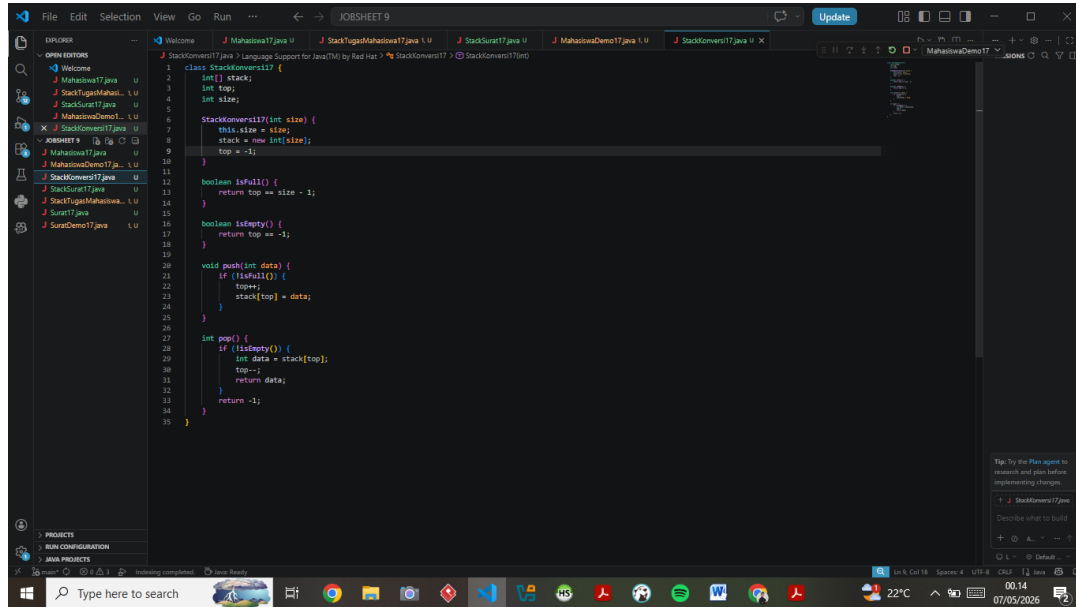
```
int jumlahTugas() {  
    return top + 1;  
}
```

Penjelasan:

- top mulai dari -1
- jadi jumlah data = `top + 1`

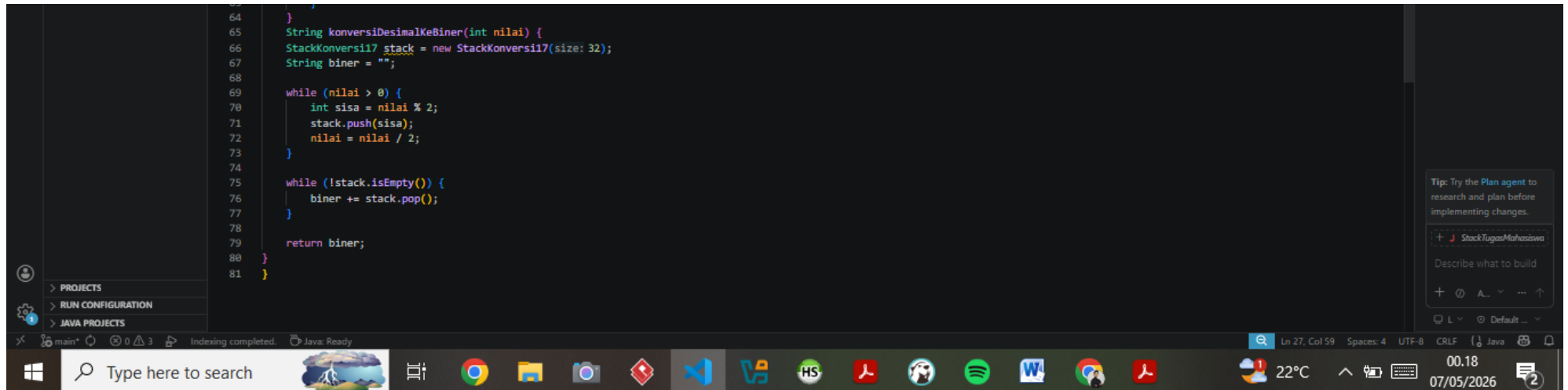
PERCOBAAN 2 – Konversi Nilai ke Biner

Class StackKonversi



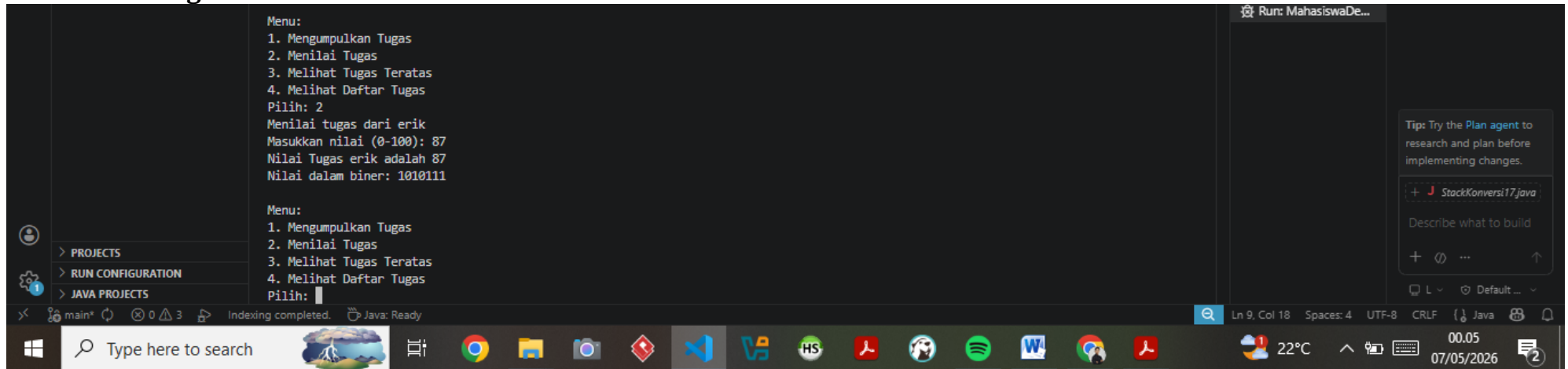
```
1 class StackKonversi17 {
2     int[] stack;
3     int top;
4     int size;
5
6     StackKonversi17(int size) {
7         this.size = size;
8         stack = new int[size];
9         top = -1;
10    }
11
12    boolean isFull() {
13        return top == size - 1;
14    }
15
16    boolean isEmpty() {
17        return top == -1;
18    }
19
20    void push(int data) {
21        if (!isFull()) {
22            top++;
23            stack[top] = data;
24        }
25    }
26
27    int pop() {
28        if (!isEmpty()) {
29            int data = stack[top];
30            top--;
31            return data;
32        }
33        return -1;
34    }
35 }
```

Method Konversi



```
64 }
65
66 String konversiDesimalKeBiner(int nilai) {
67     StackKonversi17 stack = new StackKonversi17(size: 32);
68     String biner = "";
69
70     while (nilai > 0) {
71         int sisa = nilai % 2;
72         stack.push(sisa);
73         nilai = nilai / 2;
74     }
75
76     while (!stack.isEmpty()) {
77         biner += stack.pop();
78     }
79
80     return biner;
81 }
```

Hasil Run Program



The screenshot shows an IDE window with a dark theme. The main editor area displays the output of a Java program. The output consists of two identical menu screens. The first menu screen shows a list of tasks: 1. Mengumpulkan Tugas, 2. Menilai Tugas, 3. Melihat Tugas Teratas, and 4. Melihat Daftar Tugas. The user selects option 2, and the program prompts for a task name and a score. The user enters 'erik' and '87', and the program outputs 'Nilai Tugas erik adalah 87' and 'Nilai dalam biner: 1010111'. The second menu screen is identical. The sidebar on the left shows the 'PROJECTS' section with 'PROJECTS', 'RUN CONFIGURATION', and 'JAVA PROJECTS' listed. The bottom status bar shows 'Ln 9, Col 18', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Java'. The Windows taskbar at the bottom shows the search bar, task view, and various application icons.

```
Menu:  
1. Mengumpulkan Tugas  
2. Menilai Tugas  
3. Melihat Tugas Teratas  
4. Melihat Daftar Tugas  
Pilih: 2  
Menilai tugas dari erik  
Masukkan nilai (0-100): 87  
Nilai Tugas erik adalah 87  
Nilai dalam biner: 1010111  
  
Menu:  
1. Mengumpulkan Tugas  
2. Menilai Tugas  
3. Melihat Tugas Teratas  
4. Melihat Daftar Tugas  
Pilih: 
```

Penjelasan

Proses:

1. angka dibagi 2
2. sisa disimpan
3. hasil dibalik dengan stack

Contoh 10:

- $10 \% 2 = 0$
- $5 \% 2 = 1$
- $2 \% 2 = 0$
- $1 \% 2 = 1$

Hasil: 1010

Jawaban Pertanyaan Percobaan 2

1. Alur method konversi

Bilangan dibagi 2 berulang.

Sisa tiap pembagian dimasukkan stack.

Karena stack LIFO, hasil menjadi urutan biner benar.

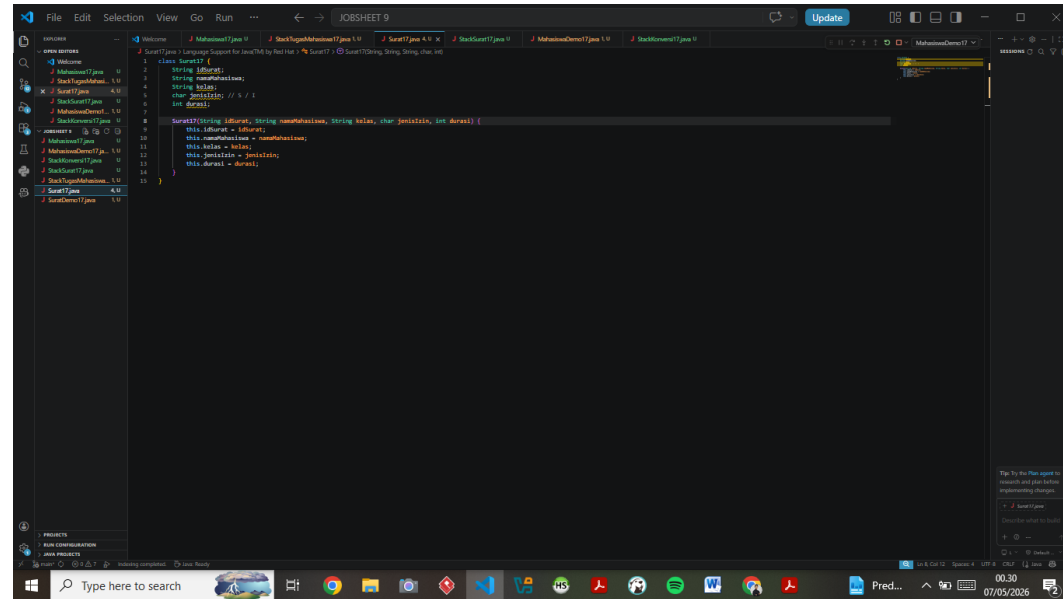
2. Jika while(kode != 0)

Program tetap berjalan normal.

Lebih jelas karena berhenti saat nilai benar-benar 0.

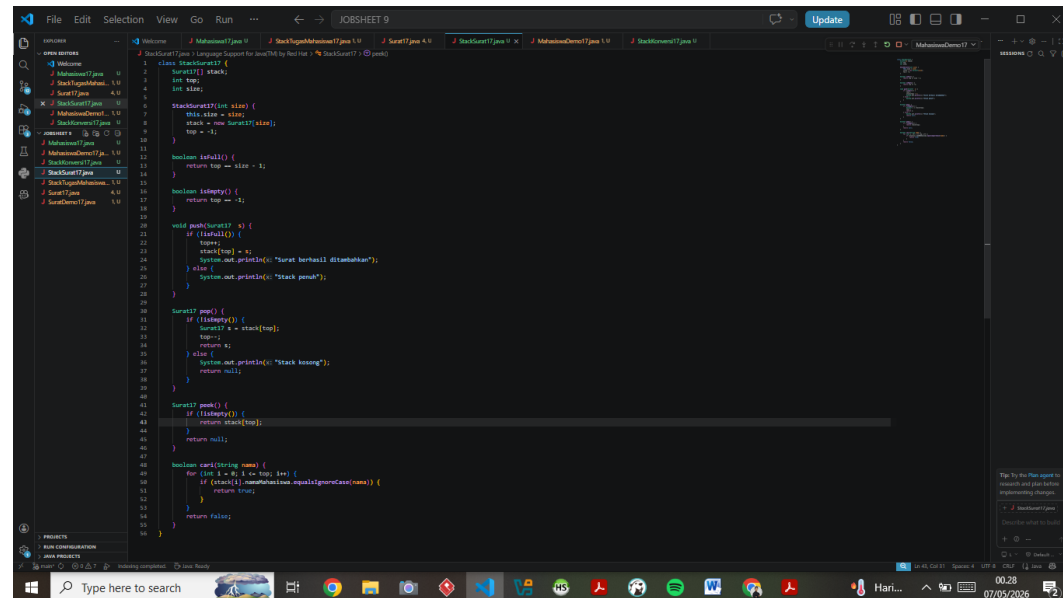
E. Latihan Praktikum – Surat Izin

Class Surat



```
1 class Surat {
2     String idSurat;
3     String namaMabaikan;
4     String kelas;
5     char jenisJenis; // S / I
6     int umur;
7
8     Surat(String idSurat, String namaMabaikan, String kelas, char jenisJenis, int umur) {
9         this.idSurat = idSurat;
10        this.namaMabaikan = namaMabaikan;
11        this.kelas = kelas;
12        this.jenisJenis = jenisJenis;
13        this.umur = umur;
14    }
15 }
```

Class StackSurat



```
1 class StackSurat {
2     Stack<Surat> stack;
3     int top;
4     int size;
5
6     StackSurat(int size) {
7         this.size = size;
8         stack = new Stack<Surat>(size);
9         top = -1;
10    }
11
12    boolean isEmpty() {
13        return top == size - 1;
14    }
15
16    boolean isFull() {
17        return top == size - 1;
18    }
19
20    void push(Surat s) {
21        if (isFull()) {
22            System.out.println("Stack penuh");
23            return;
24        }
25        stack.add(s);
26        top++;
27    }
28
29    Surat pop() {
30        if (isEmpty()) {
31            System.out.println("Stack kosong");
32            return null;
33        }
34        Surat s = stack.get(top);
35        stack.remove(top);
36        top--;
37        return s;
38    }
39
40    Surat peek() {
41        if (isEmpty()) {
42            return null;
43        }
44        return stack.get(top);
45    }
46
47    boolean cari(String nama) {
48        for (int i = 0; i < size; i++) {
49            if (stack.get(i).namaMabaikan.equals(nama)) {
50                return true;
51            }
52        }
53        return false;
54    }
55 }
```

Class SuratDemo

The image shows a Windows desktop environment. In the foreground, a Visual Studio Code editor window is open, displaying a Java file named 'JORSHEET9.java'. The code is a Java program that simulates a stack using an array. It includes a 'main' method that creates a 'Stack' object, pushes elements, and then pops them, printing the stack's state and the popped elements. The code is written in a standard Java syntax with comments in Indonesian. The background shows a Windows taskbar with various application icons and a date/time display in the bottom right corner showing '07/05/2026'.

Hasil Run Program

[illegible]

Penjelasan

Program digunakan untuk:

- menerima surat
- memproses surat
- melihat surat terakhir
- mencari surat mahasiswa

Menggunakan stack karena surat terakhir diproses lebih dulu.

Kesimpulan

1. Stack menggunakan konsep LIFO.
2. Stack cocok untuk data bertumpuk.
3. Operasi stack:
 - push
 - pop
 - peek
 - isFull
 - isEmpty
4. Stack mempermudah pengelolaan tugas, surat izin, dan konversi biner.

Link GitHub: <https://github.com/saumsaifulloh/AlgoritmaDanStrukturDataSem2/tree/main/ASD/PASD/JOBSHEET%209>